

ClipSync AI

Your clipboard, with a brain — and nothing ever leaves your machine.
Product documentation: what the app does, how it keeps your data yours, and how to build and verify it yourself.

Contents

1 Why this exists	3
At a glance	3
2 Two apps, one repo, zero cloud	3
3 What it does, in plain words	3
Clips	3
Chat, Notes, Capture	4
Privacy controls	4
4 How your stuff stays yours	4
Honest limits	5
5 Everyday keys (Windows)	5
6 Build it yourself	5
Android	5
Windows	5
Check the work	5
Linux and macOS (GitHub Actions)	6
7 Project map	6

1 Why this exists

Every “smart clipboard” app takes the most sensitive thing on your device — your clipboard, full of passwords, one-time codes, half-written messages, medical notes — and ships it off to somebody else’s server. That is backwards. ClipSync AI does all of its thinking right there on your own hardware.

No account. No server. No telemetry. No “we value your privacy” page that means nothing. Just an app that works with the Wi-Fi off.

At a glance

What you get	Needs
Full app: clips, chat, notes, OCR, voice, share sheet, quick tile	4 GB RAM, 1–5 GB for models
Single no-installer program (~23 MB), tray, global hotkeys, history popup	.NET Framework 4.0+
.deb / .rpm / portable tarball via CI	GTK 3, Secret Service, zenity
.dmg / .zip for Apple Silicon and Intel via CI	Right-click → Open on first launch

2 Two apps, one repo, zero cloud

The repository builds two separate programs. Same idea, each one native to its home:

- Android, Linux and macOS — one Flutter app. It loads a GGUF model straight into llama.cpp inside its own process. Your device becomes the AI server: Qwen 0.5B up to Llama 7B, depending on RAM.
- Windows — a native C# program. A single no-installer executable that carries its own llama.cpp runtime inside it. Pick a GGUF from the catalogue or import your own; with no model at all, a built-in engine still cleans, classifies and tags every clip.
- Neither one has ever sent a single clip anywhere. That is not a setting — there is simply no code that does it.

3 What it does, in plain words

Clips

- Automatic capture — everything you copy is kept, with a quiet background service on Android and a tray watcher on Windows.

- Search that understands — keyword search always works, ranked by relevance (titles beat tags, tags beat bodies, rare words beat common ones). Load the small indexing model and the feed ranks by meaning: “that AWS key from last week” finds the key.
- Titles and tags at capture — written by the model when one is loaded, by keywords otherwise.
- Join clips — pick two or more and merge them into one, oldest first, with a divider between parts.
- Redact secrets — one tap masks API keys, tokens, passwords, JWTs and card numbers; auto-redact masks them before a clip is ever saved.
- Duplicate sense — saving something twice asks: keep both, merge into the earlier clip, or discard.
- Clean up, summarise, retile, pin, copy — with undo where it counts.

Chat, Notes, Capture

- Chat — a real streamed conversation with the on-device model. Attach documents, OCR-read photos, or transcribed voice notes. Cloud providers (OpenAI, Anthropic, Gemini and others) are available only through an explicit, twice-confirmed opt-in.
- Notes — a markdown editor with tags, search, and an AI bar (summarise, expand, fix grammar, action items), every action undoable.
- Capture — camera OCR, screen-region capture, dictation. On Android: share text into Clips from any app’s share sheet, or file the clipboard from a Quick Settings tile. On Windows: Ctrl+Shift+H opens a history popup that pastes back into whatever you are working in.

Privacy controls

- App lock — a PIN over the whole app, asked on launch and on every return from the background. Fingerprint or face unlocks it where the hardware exists (Android, macOS).
- Clip lifetime — keep clips forever, a day, a week, or a month. Expired, unpinned clips burn on launch. Pinned clips are never dropped.
- Export or wipe — everything out as markdown or JSON, or gone with confirmation.

4 How your stuff stays yours

- On-device inference. The model file lives in your app folder. Words go in, answers come out, nothing in between touches a network.
- Encrypted storage. AES-256 throughout — clips, notes, chats, settings, each with its own key. Android keys live in the system keystore; Windows keys are locked to your Windows account.

- Secrets never land. Auto-redact, the refuse-to-save filter, per-clip redaction, and lifetimes compose into real defence in depth.
- The ten-second audit. Turn on airplane mode (or pull the PC's network cable). Everything keeps working except model downloads.

Honest limits

- The Windows build keeps PIN-only locking: Windows Hello needs the full Windows SDK to compile against, which the offline build deliberately avoids.
- The Windows build ranks search by keywords; meaning vectors need an embedding runtime its native engine does not expose.
- Fresh unsigned builds can startle antivirus — a clipboard watcher with hotkeys sounds like spyware until you read the code.

5 Everyday keys (Windows)

Shortcut	Does
Ctrl+Shift+V	Summon the window
Ctrl+Shift+C	Tidy the clipboard in place
Ctrl+Shift+H	History popup — pick a clip, Enter pastes it back

All three are re-recordable in Settings — press the keys, no typing required.

6 Build it yourself

Android

```
flutter pub get
flutter build apk --release # → build/app/outputs/flutter-apk/
```

Windows

You need... nothing. The C# compiler already lives inside Windows:

```
windows\build.cmd # tests first, then x86 + x64 exes
```

Check the work

```
flutter analyze && flutter test # 249 Flutter tests
windows\build.cmd tests # 600 desktop tests
```

Linux and macOS (GitHub Actions)

Desktop binaries build entirely in the cloud via `.github/workflows/desktop.yml` — push to main or run the workflow by hand, then download the `.deb`, `.rpm`, `.tar.xz`, `.dmg` and `.zip` artifacts from the green run.

7 Project map

Path	What lives there
<code>lib/main.dart</code>	The Flutter app: clips feed, chat, notes, capture, settings, lock screen
<code>lib/services/</code>	On-device LLM, embeddings, biometrics, storage, catalogues
<code>lib/features/</code> + <code>lib/ui/</code>	History, sheets, and the Liquid Glass design system
<code>windows/src/</code>	The native Windows app (C# / WinForms)
<code>windows/tests/</code>	The desktop test suite
<code>android/</code>	Native shell: service, share sheet, quick tile
<code>linux/</code> <code>macos/</code>	Desktop Flutter scaffolding
<code>test/</code>	The Flutter test suite
<code>.github/workflows/</code>	Linux + macOS build pipeline